Lecture 6 - Analysis and Efficiency

Disclaimer: This might be one of the hardest things we have to cover. If you are unsure ask an instructor in the tutorials. When you first see this it isn't obvious.

Experimental Analysis

We can study the running time of any algorithm by timing its execution time.

We can determine the elapsed time by using the python `time` module. There is a more sophisticated module called `timeit`, but we won't use that for now.

```
import time
start = time.perf_counter()     # start the timer

# run the code to be timed here

finish = time.perf_counter()    # stop the timer
elapsed_time = finish - start   # calculate the elasped time
```

Be warned that because many processes share the computer's central processing unit (CPU), the elapsed time is an approximation. Really, you should run this many times and average the result.

For the type of analysis we are interested in, this is normally not necessary as we will look at the behaviour of the algorithm for different values of its input size and the pattern is normally fairly obvious.

Here are two example pieces of code that compute what is known as the prefix (cumulative) average. You can find an animation explaining the prefix average in the lecture content and it was demoed in the lecture.

Code Example 1

```
def prefix_average1(S):
    """Return list such that, for all j, A[j] equals average of S[0], ...,
S[j]."""
    n = len(S)
    A = [0]*n                   # create new list of n zeros
    for j in range(n):
        total = 0               # begin computing S[0] + ... + S[j]
        for i in range(j + 1):
            total += S[i]
        A[j] = total / (j+1)    # record the average
    return A
```

You can run this code to see how it works on Python Tutor

Code Example 2

```
def prefix_average2(S):
    """Return list such that, for all j, A[j] equals average of S[0], ...,
S[j]."""
    n = len(S)
    A = [0]*n                   # create new list of n zeros
    total = 0                   # compute prefix sum as S[0] + S[1] + ...
    for j in range(n):
        total += S[j]           # update prefix sum to include S[j]
        A[j] = total / (j+1)    # compute average based on current sum
    return A
```

You can run this code to see how it works on Python Tutor

Analysing the Examples

You can see from this analysis that for large lists, that is when their length n gets large, `prefix_average2()` is faster than `prefix_average1()`.

You will notice some fluctuations in this, this is because the timing is not exact. In fact, you are sharing a central processing unit (CPU) with lots of other processes on the machine. Sometimes the CPU will wander off and do something else as a priority.

Issues With Experimental Analysis

Whilst experimental analysis is extremely useful, it does have some major limitations:

- We can't test for all possible inputs (it could be that some of the values we have missed out are important).
- Comparing algorithms requires us to run experiments using the same hardware and software.
- An algorithm has to be implemented to be tested!

The last point is fundamental, if you require an algorithm to be implemented to analyse how it performs, how can you design an algorithm to be efficient in the first place.

Beyond Experimental Analysis

What we need is a way to analyse algorithms that:

1. Does not require the implementation of the algorithm.
2. Analysis is independent of hardware and software.
3. We consider all inputs.

This is the purpose of this lecture. We will look at how we can analyse an algorithm by counting the number of ***basic operations*** it takes.

We must define what we mean by ***basic operations***

Model of Computation

We will use the word-RAM (Random Access Memory) model of computation which gives us access to memory consisting of cells. Each cell contains a machine word. We can read and write to these cells and perform a set of ***basic operations*** which will correspond to a low-level instruction in constant time. You can read more about the word-RAM model in a book or online if you are interested.

Be warned, it is a model, it is not an exact representation of the machine, but it is good enough for us to use for analysis. It is about finding a balance between simplicity and completely imitating the machine.

The following will be **basic operations** that will take constant time:

1. Arithmetic operations such as `+,-,\,*`
2. Comparisons operations such as `<,>,<=,>=, ==`
3. Boolean operations such as `and`, `or` and `not`
4. Reading and writing a word to memory

We donate constant time by $O(1)$. This will make more sense later on.

Counting Basic Operations Based on Input Size

We will count the number of operations based on the input size. We can count operations as follows:

```
for i in range(10):     # 10 - how many times the loop runs
    print(i)            # O(1)
```

In this case, we have $10 \cdot O(1)$. That is 10 basic operations (remember basic operations take constant time).

KEY POINT: Therefore this takes constant time! i.e. $O(1)$.

Above, read $10 \cdot O(1)$ as multiplication, it is the same as $10 \times O(1)$.

Worse Case Analysis

The **worst-case time complexity** indicates the longest running time performed by an algorithm given any input of size n, and thus guarantees that the algorithm will finish in the indicated period of time.

Average Case Analysis

In general, we will focus on worse-case analysis. Computing average-case analysis is much more complicated.

Big-O Analysis

In this section we will introduce big-O notation and analysis. The need for this is so that we have a standard technique and language by which we can compare algorithms irrespective of what machine we run them on. We only care about how they perform as the input size n grows.

Growth Rates

When analysing algorithms it is key to understand the growth (cost) of the algorithm as n gets larger. Therefore we need to understand some key functions and how they compare with one another.

7 Key Functions

Before we learn about the famous (and somewhat confusing) big-O notation, we need to remind ourselves of some important functions.

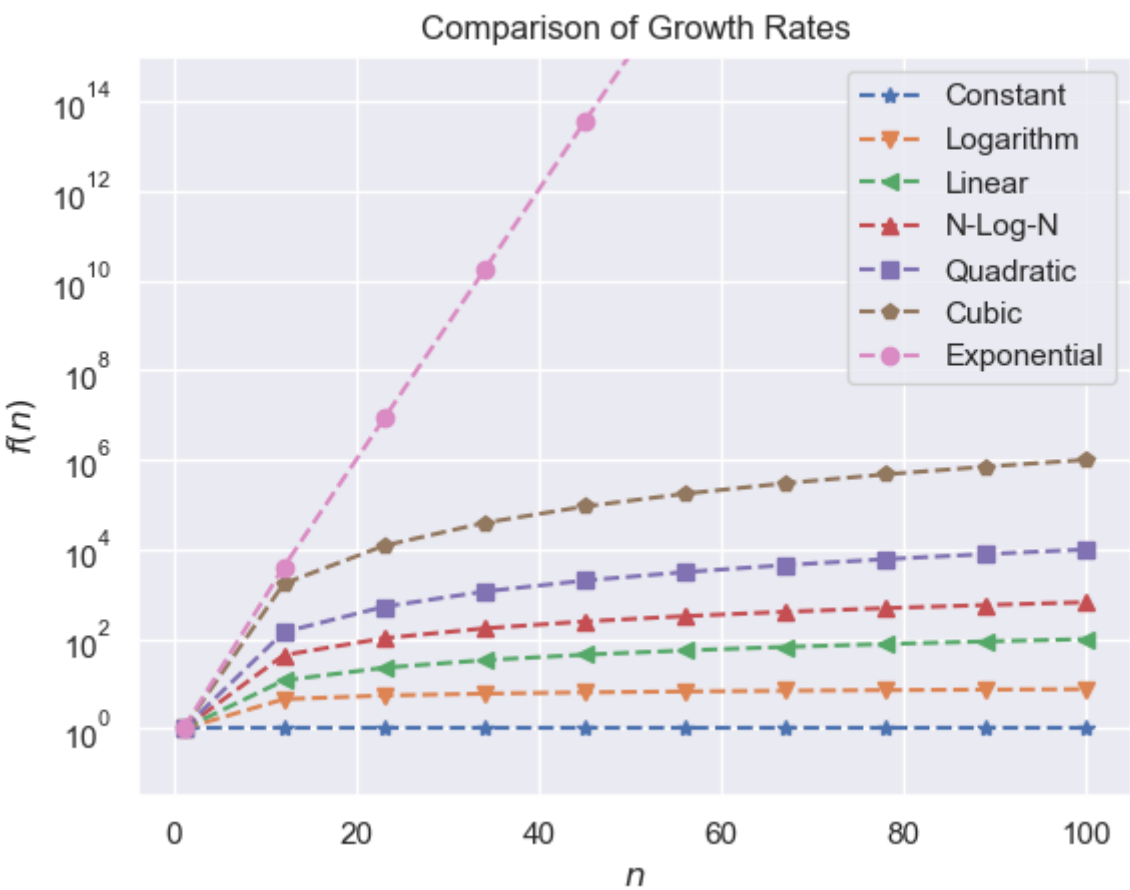
Here is a table summary, but I've provided more explanation below. These are listed from best to worse.

Name	$f(n)$	**Example**
Constant	$f(n) = c$	$f(n) = 5$
Logarithm	$f(n) = \log_b(n)$ we will mostly use base 2 and denote: $\log(n) = \log_2(n)$	$f(n) = \log_{10}(n)$ $f(n) = \log(n)$

Name	$f(n)$	Example
Linear	$f(n) = n$	
N-Log-N	$f(n) = n\log(n)$	
Quadratic	$f(n) = n^2$	
Cubic	$f(n) = n^3$	
Polynomial	$f(n) = a_0 + a_1 n + a_2 n^2 + a_3 n^3 + \cdots + a_d n^d$ where $a_0, a_1, \ldots, a_d$ are constants, called the coefficients of the polynomial, and $a_d \neq 0$. Integer d is called the degree of the polynomial.	$f(n) = 4n^3 + 2n$ $f(n) = 4n^4 + 2n^2 + 4n$
Exponential	$f(n) = b^n$ where b is a positive constant.	$f(n) = 2^n$ $f(n) = 4^n$

Comparison of Growth Rates

The following plot compares how as n gets larger these functions compare with each other. This is critical in understanding why we want functions that don't act exponentially.



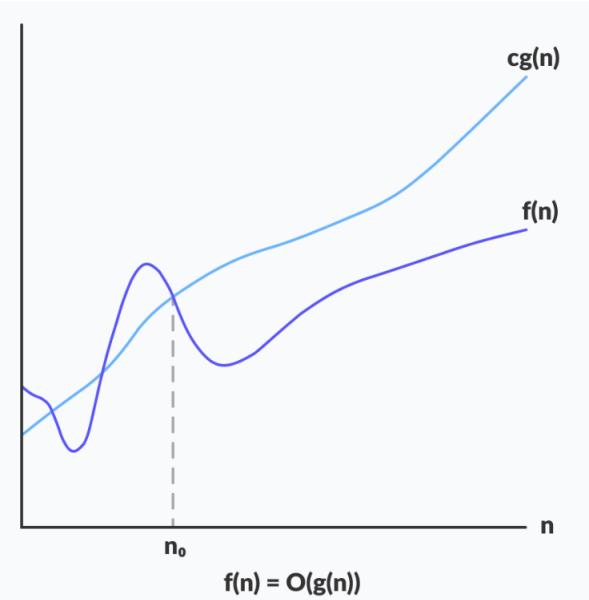
There is also a nice plot on the big-O cheat sheet website.

Big-O Notation

Big-O notation is a way of providing an upper bound on the number of **basic operations** an algorithm performs based on an input parameter n. This is always the size of the problem.

> ### Formal Definition
>
> Let $f(n)$ and $g(n)$ be functions that map positive integers to real numbers, i.e. $f : \mathbb{Z} \to \mathbb{R}$.
>
> $f(n)$ is $O(g(n))$ if there is a real constant c and an integer constant $n_0 \geq 1$ such that
>
> $$f(n) \leq cg(n), \quad \text{for } n \geq n_0$$

You can think about this graphically as follows.



Programviz Graph

What does that even mean!?

In plain English, the function $f(n)$ is said to be $O(g(n))$ (big-O $g(n)$) if for a big enough value of n (we call that n_0) the function $f(n)$ is always smaller than $cf(n)$.

For example, if we are looking for the maximum item in a list of numbers. n is the length of the list.

The idea is that we count the number of **basic operations** and then look at the maximum growth rate.

Example 1

The function $10n + 6$ is $O(n)$.

We have to find a constant $c > 0$ and $n_0 \geq 1$ such that

$$10n + 6 \leq cn, \quad \text{for } n \geq n_0$$

Pick $c = 11$ and $n_0 = 6$.

We are stating

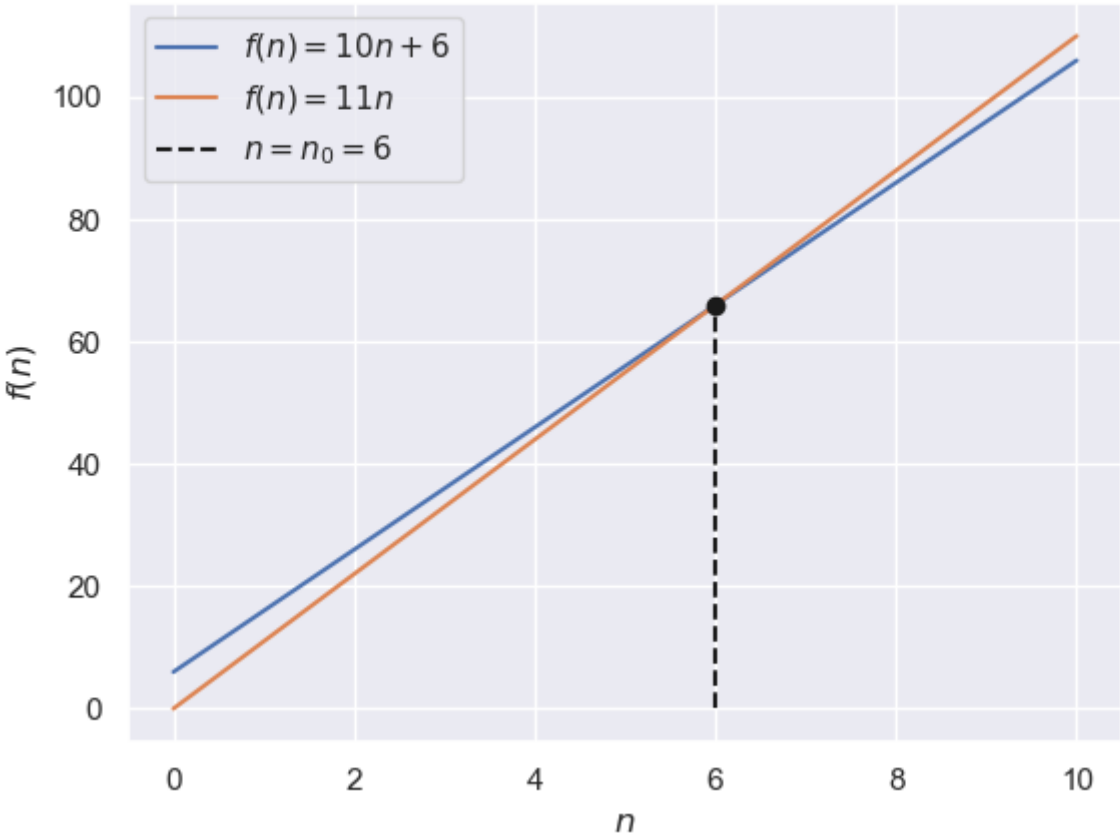
$$10n + 6 \leq 11n, \quad \text{for } n \geq 6$$

Is this true?

Try for $6, 7, 8, \ldots$

n	$10n + 6$	$11n$	$10n + 6 \leq 66$
5	56	55	`False`
6	66	66	`True`
7	76	77	`True`
8	86	88	`True`

Plot of $10n + 6$ and $11n$



Example 2: An Easier Way

Let's say we counted the total number of **basic operations** in an algorithm to be

$$3n^2 + 2n$$

then n^2 is the dominant term. By that, we mean that as n gets bigger and bigger, i.e. $n \to \infty$, n^2 is the term that affects the total the most.

Therefore we say that the worse case running time is $O(n^2)$

Other Examples

Here is a table showing the number of basic operations and the big-O running time.

Number of Basic Operations	Big-O Running Time
$4n + 3$	$O(n)$
$5n^2 + 4$	$O(n^2)$
$5n^4 + 34n^2 + 4$	$O(n^4)$
$2^n + 3n^4 + n^2$	$O(2^n)$
$n\log(n) + 5n$	$O(n\log(n))$

Analysis of Algorithms

We are now ready to analyse some algorithms using big-O notation.

Counting Operations Based on Input Size

Let's return to our first counting example. We can count operations as follows.

```
for i in range(10):     # 10 - how many times the loop runs
    print(i)            # O(1)
```

In this case, we have $10 \cdot O(1)$. That is 10 basic operations (remember basic operations take constant time).

KEY POINT: Therefore this takes constant time! i.e. $O(1)$.

Above, read $10 \cdot O(1)$ as multiplication, it is the same as $10 \times O(1)$.

If we take an arbitrarily large number n of numbers to print out. Note to do this we have wrapped the previous code in a function. It is now easy to see the input is the parameter `n`.

```
def print_num(n)
  for i in range(n):      # n - how many times the loop runs
      print(i)            # O(1)
```

We have $n \cdot O(1)$ ***basic operations***. That is $O(n)$. That is, we have an upper bound of n basic operations.

KEY POINT: What this means is the number of operations is no longer constant, it is now dependent on the value of n.

Example: Analysis of `prefix_average1()`

```
def prefix_average1(S):
    """Return list such that, for all j, A[j] equals average of S[0], ...,
S[j]."""
    n = len(S)                      # O(1) - Python stores the length so retrieval
is constant time
    A = [0]*n                       # O(n) - Create a list with n items
    for j in range(n):              # n - loops n times
        total = 0                   # O(1)
        for i in range(j + 1):      # j+1 - loops j+1 times
            total += S[i]           # O(1)
        A[j] = total / (j+1)        # O(1) - Technically there are two operations
here, but that is just constant time!
    return A
```

- `n=len(S)` contributes constant time, 1 basic operation. $O(1)$
- `A = [0]*n` takes n operations, this is $O(n)$ time.
- The outer for loop `for j in range(n)` is executed n times. Each loop we execute the statements `total=0` and `A[j] = total / (j+1)`. So this is n lots of 3 basic operations, so $3n$. This contributes $O(n)$.
- The inner loop `for i in range(j + 1)` executes 1 basic operation `total += S[i]`. As it is in the outer loop, it executes `j+1` times for each value of `j`, that is $1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$ times. Therefore this contributes $O(n^2)$.

In total we have $O(1) + O(n) + O(n) + O(n^2)$, as n gets bigger and bigger $O(n^2)$ dominates. Thus our time complexity for this algorithm is $O(n^2)$.

KEY POINT: The time of our algorithm is dependent on n and the time is proportional to n^2.

Example: Analysis of `prefix_average2()`

```
def prefix_average2(S):
    """Return list such that, for all j, A[j] equals average of S[0], ...,
S[j]."""
    n = len(S)                  # O(1)
    A = [0]*n                   # O(n)
    total = 0                   # O(1)
    for j in range(n):          # n - loops n times
        total += S[j]           # O(1)
        A[j] = total / (j+1)    # O(1)
    return A
```

- Initialising `n` and `total` takes $O(1)$ time.
- Initialising `A` takes $O(n)$.
- The single loop executes n times, and each statement in the loop contributes 1 basic operation, so $2n$ basic operations which is $O(n)$ time.
    - Or you can think that the 2 operations take constant time $O(1)$ and it is repeated n times, that is still $O(n)$.

In total we have $O(1) + O(n) + O(n)$ which is $O(n)$ time.

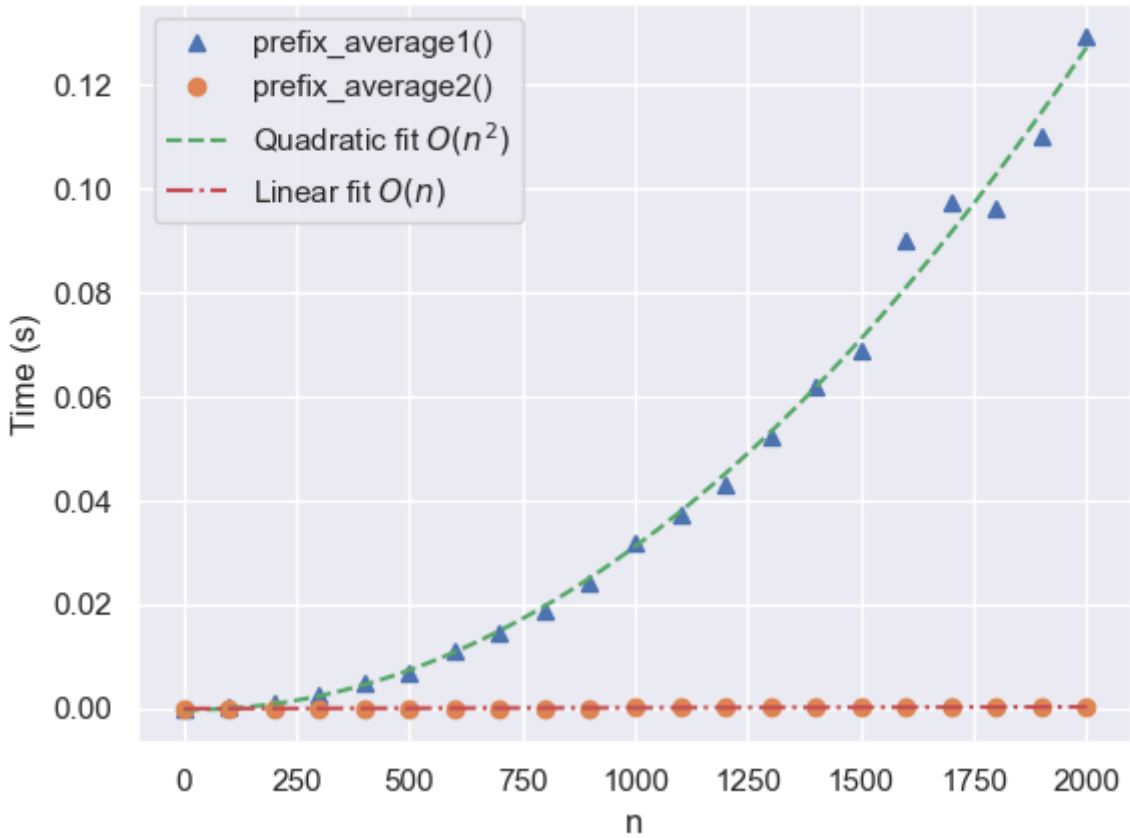
KEY POINT: The time of our algorithm is dependent on n and the time is proportional to n.

Comparison of `prefix_average1()` and `prefix_average2()`

So with no experimental analysis, we have worked out how prefix algorithm 1 and prefix algorithm 2 grow proportionally to n.

We have calculated that algorithm 2 is a faster algorithm with a time-complexity of $O(n)$ compared with $O(n^2)$ for algorithm 1. That is we have an idea of their growth as n gets large.

Just for completeness, here is the opening plot from the **Experimental Analysis** section. I have added two fitted lines. One is linear $O(n)$, the other is $O(n^2)$.



You can see by inspection that they agree with our big-O analysis!

Even on really amazing hardware, as n grows large, it can't save you from the poorer performance of algorithm 1.

Summary

- We can count the number of **basic operations** an algorithm performs.
- We can use worse-case analysis to understand the running time of an algorithm based on the input size n.
- We use big-O notation to provide an upper bound on this worse-case analysis.
- NOTE: We can also use big-O notation for average-case analysis and best-case analysis.

Key Takeaway

If someone tells you an algorithm has a worse-case complexity of $O(n)$, this is better than another algorithm with a worse-case complexity of $O(n^2)$.

This applies to all growth rates. $O(1) < O(\log(n)) < O(n) < O(n\log(n)) < O(n^2) < O(2^n)$.

References

- Data Structures and Algorithms in Python By Michael T. Goodrich, Michael H. Goldwasser, Roberto Tamassia
- Big-O Cheat Sheet